claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-a5f0b3f9fb58660ae.jsonl`
- SHA-256: `096e5a37e8f2694253f0f9ec4bec51255056075f2fd5b1644b61f3d23286b2ba`
- Source modified: `2026-06-10T06:08:13+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_2a64bc88-2a8`
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`

Transcript

1. user (2026-06-10T06:06:42.245Z)

CONTEXT ? two sibling repos on a Windows machine:
- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).
- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).
Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources.
ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.
QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.
ROLE: adversarial verifier. An auditor ("testing") claimed the finding below. Try to REFUTE it against the actual files. Default to isReal=false if the evidence does not hold up, the behavior is documented/deliberate (e.g. listed in docs/CODE_MAP.md gotchas or DEFERRED_HARDENING_PLAN.md as a known deferral ? unless the auditor adds genuinely NEW consequence), or the severity is materially overstated.
FINDING: {"title":"ffmpeg stitcher (VideoCompiler) completely untested ? /api/compile path has no coverage end to end","severity":"high","area":"stitcher / compiler","file":"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/stitcher/compiler.py (221 lines)","evidence":"compiler.py is absent from the coverage report (never imported by any test). It contains pure-logic that needs no ffmpeg to test: _parse_time (line 17), duration clamping, per-segment ffmpeg command construction (line 138), CalledProcessError skip-and-continue with stderr tail capture (lines 158-163), and concat-list building (line 194). Its only import-time guard is the subprocess smoke test docstring in tests/test_import_smoke.py:5-6 ('the stitcher compiler, which only backend.main pulls in, used to go unnoticed') ? acknowledging it previously shipped broken.","why_it_matters":"The compile/stitch step is the user-visible deliverable (the highlight reel). Command-construction bugs (quoting, -ss/-to ordering, padding arithmetic, segments out of range) would only be discovered on a real machine run; subprocess.run is trivially mockable so the gap is not forced by the no-GPU constraint. The repo's own smoke-test docstring records this exact file shipping with an unnoticed break before."}
Steps: open the cited file(s) and any counter-evidence files; verify quotes/line claims; judge severity honestly. Cheap greps allowed; do NOT run the pipeline. Return isReal, confidence, and a 1-3 sentence verdict (with corrected detail if partially right).

2. user (2026-06-10T06:06:49.459Z)

```
      import os
      import subprocess
      import tempfile
      import logging
      from typing import List, Tuple, Union

      logger = logging.getLogger(__name__)

      class VideoCompiler:
          def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
0.5):
              self.output_dir = output_dir
              self.end_padding = end_padding
              self.begin_padding = begin_padding
              os.makedirs(output_dir, exist_ok=True)

          @staticmethod
          def _parse_time(val: Union[int, float, str]) -> float:
              """Normalizes a timestamp value to float seconds.
              Handles: float/int (pass-through), plain numeric strings ("12.4"),
              and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
              This mirrors the parse_time logic used in gemini.py to ensure
              the stitcher can always consume whatever format the indexer produced."""
              if isinstance(val, (int, float)):
                  return float(val)
              if isinstance(val, str):
                  val = val.strip()
                  if ":" in val:
                      # Handle MM:SS, HH:MM:SS, etc.
                      parts = val.split(":")
                      parts.reverse()
                      total = 0.0
                      for i, p in enumerate(parts):
                          try:
                              total += float(p) * (60 ** i)
                          except ValueError:
                              pass
                      return total
                  try:
                      return float(val)
                  except ValueError:
                      logger.warning(f"Could not parse timestamp value '{val}' as a number.
Defaulting to 0.0.")
                      return 0.0
              logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
'{val}'. Defaulting to 0.0.")
              return 0.0

          def _get_video_duration(self, video_path: str) -> float:
              """Probes the video file to get its total duration in seconds.
              Returns 0.0 if it cannot be determined."""
              try:
                  result = subprocess.run(
                      [
                          "ffprobe", "-v", "error",
                          "-show_entries", "format=duration",
                          "-of", "default=noprint_wrappers=1:nokey=1",
                          video_path
                      ],
                      capture_output=True, text=True, check=True
                  )
                  return float(result.stdout.strip())
              except Exception as e:
                  logger.warning(f"Could not determine video duration via ffprobe: {e}")
                  return 0.0
```

```
63
64          def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float,
float]], output_filename: str) -> str:
65              """
66              Extracts selected segments from a raw video and stitches them together
67              into a seamless highlights file, applying end_padding to prevent abrupt endings.
68              """
69              if not segments:
70                  raise ValueError("No highlight segments provided to compile.")
71
72              if not os.path.exists(raw_video_path):
73                  raise FileNotFoundError(f"[COMPILER ERROR] Source video file not found:
{raw_video_path}")
74
75              output_path = os.path.join(self.output_dir, output_filename)
76
77              # Probe video duration so we can detect out-of-range segments
78              video_duration = self._get_video_duration(raw_video_path)
79              if video_duration > 0:
80                  logger.info(f"Source video duration: {video_duration:.2f}s")
81              else:
82                  logger.warning("Video duration unknown. Cannot validate segment boundaries
against video length.")
83
84              # We will create temporary clips and stitch them using FFmpeg concat demuxer
85              temp_clips = []
86              skipped_segments = []
87
88              # Create a temp directory within output_dir
89              with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
90                  logger.info(f"Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
91                  for idx, (raw_start, raw_end) in enumerate(segments):
92                      # Normalize timestamps to float seconds ? handles float, int,
93                      # plain numeric strings, and colon-separated formats like "MM:SS"
94                      start = self._parse_time(raw_start)
95                      end = self._parse_time(raw_end)
96                      segment_label = f"Segment #{idx+1}/{len(segments)} [{start:.2f}s -
{end:.2f}s]"
97
98                      # Validate the segment times
99                      if start < 0:
100                         logger.warning(f"{segment_label}: start_time is negative
({start:.2f}s). Clamping to 0.")
101                         start = 0.0
102
103                     if start >= end:
104                         logger.error(f"{segment_label}: start_time ({start:.2f}s) >=
end_time ({end:.2f}s). Skipping invalid segment.")
105                         skipped_segments.append((idx, start, end, "start >= end"))
106                         continue
107
108                     # Check if segment extends beyond video duration
109                     if video_duration > 0:
110                         if start >= video_duration:
111                             logger.error(f"{segment_label}: start_time ({start:.2f}s) is
beyond the video duration ({video_duration:.2f}s). "
112                                          f"This segment cannot be extracted ? the video ran
out. Skipping.")
113                             skipped_segments.append((idx, start, end, "start beyond video
end"))
114                             continue
115
116                         if end > video_duration:
117                             original_end = end
```

```
                            end = video_duration
                            logger.warning(f"{segment_label}: end_time ({original_end:.2f}s)
exceeds video duration ({video_duration:.2f}s). "
                                           f"Clamping end_time to {video_duration:.2f}s. The
segment may be truncated.")

                    clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")

                    # Precise sub-second trim using fast re-encoding to preserve stream
headers
                    padded_start = max(0.0, start - self.begin_padding)
                    padded_end = end + self.end_padding

                    # Clamp padded_end to video duration if known
                    if video_duration > 0 and padded_end > video_duration:
                        padded_end = video_duration
                        logger.info(f"{segment_label}: Padded end ({end +
self.end_padding:.2f}s) exceeds video duration. "
                                    f"Clamping to {video_duration:.2f}s (end padding
partially applied).")

                    trim_duration = padded_end - padded_start
                    logger.info(f"Trimming {segment_label} -> [{padded_start:.3f}s -
{padded_end:.3f}s] (duration: {trim_duration:.2f}s)")

                    cmd = [
                        "ffmpeg", "-y",
                        "-ss", str(round(padded_start, 3)),
                        "-to", str(round(padded_end, 3)),
                        "-i", raw_video_path,
                        "-c:v", "libx264",
                        "-preset", "superfast",
                        "-crf", "22",
                        "-c:a", "aac",
                        "-b:a", "128k",
                        clip_path
                    ]

                    try:
                        result = subprocess.run(cmd, capture_output=True, check=True)
                        # Verify the output clip was actually created and has content
                        if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
                            temp_clips.append(clip_path)
                        else:
                            logger.error(f"{segment_label}: FFmpeg exited successfully but
output clip is missing or empty. Skipping.")
                            skipped_segments.append((idx, start, end, "empty output clip"))
                    except subprocess.CalledProcessError as e:
                        stderr_msg = e.stderr.decode(errors='replace').strip()
                        logger.error(f"{segment_label}: FFmpeg trim failed.")
                        print(f"  Command: {' '.join(cmd)}")
                        print(f"  FFmpeg stderr: {stderr_msg[-500:]}")  # Last 500 chars to
avoid flooding
                        skipped_segments.append((idx, start, end, f"ffmpeg error:
{stderr_msg[-200:]}"))
                        continue

                # Report summary of skipped segments
                if skipped_segments:
                    logger.info(f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:")
                    for seg_idx, seg_start, seg_end, reason in skipped_segments:
                        logger.info(f"  - Segment #{seg_idx+1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}")
                    logger.info("")
```

```
172
173                 if not temp_clips:
174                     raise RuntimeError(
175                         f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
176                         f"All {len(segments)} segments were skipped. "
177                         f"Check the errors above for details. "
178                         f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
179                     )
180
181                 logger.info(f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch...")
182
183                 # Create a list file for FFmpeg concat demuxer
184                 list_file_path = os.path.join(tmp_dir, "clips.txt")
185                 with open(list_file_path, "w") as f:
186                     for clip in temp_clips:
187                         # Escape single quotes in path
188                         safe_path = clip.replace("'", "'\\''")
189                         f.write(f"file '{safe_path}'\n")
190
191                 logger.info("Stitching clips together...")
192                 # Stitch them together without re-encoding (since they were encoded with
identical settings)
193                 concat_cmd = [
194                     "ffmpeg", "-y",
195                     "-f", "concat",
196                     "-safe", "0",
197                     "-i", list_file_path,
198                     "-c", "copy",
199                     output_path
200                 ]
201
202                 try:
203                     result = subprocess.run(concat_cmd, capture_output=True, check=True)
204
205                     # Final verification
206                     if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
207                         raise RuntimeError(
208                             f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
209                             f"but output file is missing or empty: {output_path}"
210                         )
211
212                     output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
213                     logger.info(f"Dynamic highlight reel saved: {output_path}
({output_size_mb:.1f} MB)")
214                     return output_path
215                 except subprocess.CalledProcessError as e:
216                     stderr_msg = e.stderr.decode(errors='replace').strip()
217                     raise RuntimeError(
218                         f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
219                         f"  Command: {' '.join(concat_cmd)}\n"
220                         f"  FFmpeg stderr: {stderr_msg[-500:]}"
221                     )
222
```

3. user (2026-06-10T06:06:50.853Z)

```
tests\test_config.py:91:    /api/compile and the CLI resolve the same value. Default + file
override + the
tests\test_import_smoke.py:5:broken file that nothing imports (e.g. the stitcher compiler, which
only
tests\test_import_smoke.py:14:import py_compile
tests\test_import_smoke.py:23:    """Every .py file under backend/ must byte-compile."""
tests\test_import_smoke.py:27:            py_compile.compile(str(path), doraise=True)
```

```
tests\test_import_smoke.py:28:        except py_compile.PyCompileError as exc:
tests\test_import_smoke.py:34:    """backend.main (FastAPI app, /api/compile) must be
importable.
```

4. user (2026-06-10T06:06:58.964Z)

```
1       """Import/syntax smoke tests for the backend package.
2
3       Guards against a SyntaxError (or module-level import error) shipping while the
4       suite stays green: most of the suite imports individual backend modules, so a
5       broken file that nothing imports (e.g. the stitcher compiler, which only
6       backend.main pulls in) used to go unnoticed.
7
8       Both tests are offline and dependency-light: the syntax sweep never imports
9       anything, and the backend.main import runs in a subprocess with any missing
10      heavy deps (cv2/torch/numpy) stubbed, so they also pass in environments
11      without the GPU stack.
12      """
13
14      import py_compile
15      import subprocess
16      import sys
17      from pathlib import Path
18
19      REPO_ROOT = Path(__file__).resolve().parents[1]
20
21
22      def test_backend_package_has_no_syntax_errors():
23          """Every .py file under backend/ must byte-compile."""
24          failures = []
25          for path in sorted((REPO_ROOT / "backend").rglob("*.py")):
26              try:
27                  py_compile.compile(str(path), doraise=True)
28              except py_compile.PyCompileError as exc:
29                  failures.append(f"{path.relative_to(REPO_ROOT)}: {exc.msg}")
30          assert not failures, "Syntax errors in backend/:\n" + "\n".join(failures)
31
32
33      def test_backend_main_imports():
34          """backend.main (FastAPI app, /api/compile) must be importable.
35
36          Runs in a subprocess so stubbing missing heavy deps cannot leak into this
37          pytest process; cwd=repo root puts the package on sys.path.
38          """
39          prelude = (
40              "import importlib.util, sys\n"
41              "from unittest import mock\n"
42              "for name in ('cv2', 'torch', 'numpy'):\n"
43              "    if importlib.util.find_spec(name) is None:\n"
44              "        sys.modules[name] = mock.MagicMock()\n"
45              "import backend.main\n"
46          )
47          result = subprocess.run(
48              [sys.executable, "-c", prelude],
49              cwd=str(REPO_ROOT),
50              capture_output=True,
51              text=True,
52          )
53          assert result.returncode == 0, (
54              "import backend.main failed:\n" + result.stderr.strip()
55          )
56
```

5. user (2026-06-10T06:07:00.536Z)

```
docs\BACKLOG.md:63:  - `backend/pipeline/` (validators + segmenters + stitcher + audio +
detectors)
docs\CODE_MAP.md:35:  -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name)
@backend/pipeline/stitcher/compiler.py
docs\CODE_MAP.md:81:- `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via
FLAGS). Static UI mounts `/static` from `backend/api/static`. Logging is initialized with
`basicConfig` (INFO, timestamped format); backend modules use `logger =
logging.getLogger(__name__)` (replaces stray prints in hot paths like motion, stitcher,
wasb_infer, config loader). User-facing CLI summaries may still use `print` for direct output.
docs\CODE_MAP.md:84:  - `::load_config` ? ? legacy thin wrapper returning a **dict**
(`load_app_config(path).model_dump(...)`); kept for transition ? new code imports
`load_app_config`/`AppConfig` directly. `::ProcessRequest`/`::QueryRequest`/`::CompileRequest`
pydantic bodies.
docs\CODE_MAP.md:148:### 2.4 Stitcher / tools / utils
docs\CODE_MAP.md:149:[Omitted long matching line]
docs\CODE_MAP.md:150:- `@backend/tools/rally_slicer.py` ? standalone CLI.
`::compute_clip_windows` (pure, unit-tested), `::load_rally_labels` (golden schema; ? silently
skips degenerate rows), `::slice_rallies`, `::ClipWindow`(rally:str),
`::BEGIN_PADDING=0.5`/`::END_PADDING=1.0` (? duplicate compiler's by copy). `_read_time ->
backend/eval/annotations.parse_timestamp`.
docs\CODE_MAP.md:254:- **Hardcoded tuned constants:** `calibrate_wasb.BASELINE`, `export.TUNED`,
slicer/compiler padding ? duplicated by copy, not import; re-tuning won't propagate.
docs\CODE_MAP.md:268:| Change stitching padding | `config.json
stitching.{begin_padding,end_padding}`; impl
`@backend/pipeline/stitcher/compiler.py::VideoCompiler.compile_highlights`; mirror
`@backend/tools/rally_slicer.py::BEGIN_PADDING/END_PADDING` |
docs\CODE_MAP.md:328:| (no dedicated test) | individual `pipeline/validators/*` impls,
`pipeline/stitcher.compiler`, `main.py`, `run_process_and_stitch.py`,
`pipeline/detectors/wasb_runner`, `pipeline/segmenters/{wasb_hybrid,motion}`,
`eval/audio/e1_probe` (diagnostic) ? ? exercised only indirectly |
docs\COMMERCIALIZATION.md:35:| C5 | **Output codec royalties** | **P0** | ? | Highlights are
still encoded **H.264 + AAC** (`libx264` in `backend/pipeline/stitcher/compiler.py`) ? encode-
side AVC patent exposure remains. | Switch output to **AV1 + Opus** (royalty-free) on an **Ada-
class GPU (L4/L40)** for HW encode. See §4. Decision: provision Ada-class GPU? |
docs\COMMERCIALIZATION.md:114:*Implementation lives in `backend/pipeline/stitcher/compiler.py`
(codec args).*
docs\archives\COMMERCIAL_READINESS_REVIEW.md:78:   - Clip gallery / scrubbing helpers (leverage
existing rally_slicer + compiler).
docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md:34:4. Logging
standardization + removal of stray `print()` in hot paths. ? (done in feat/logging-
standardization; central setup in main; replaced in motion.py, wasb_infer.py,
stitcher/compiler.py, config/loader.py, main internal diagnostics; user-facing output in
run_*.py and main CLI kept as print where appropriate for direct console UX. Docs updated in
CODE_MAP and this plan.)
docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md:76:?   ??? stitcher/
docs\archives\checkpoints\2026-06-logging-reporting\README.md:12:- Logging setup in
`backend/main.py` and hot paths (motion, wasb_infer, stitcher, config loader, etc.).
```

6. user (2026-06-10T06:07:21.770Z)

```
300     | `test_tracknet_runner.py` | `pipeline/detectors/tracknet_runner` (`to_wsl_mnt_path`,
`from_indexing_cfg`, parse + `trajectory_to_action_windows` shared brain, healthcheck,
run_predict, stage) |
301     | `test_rally_gate.py` | `pipeline/detectors/rally_gate` (axis/net estimate, crossing
count, gate kills single-toss) |
302     | `test_wasb_cache.py` | `pipeline/detectors/wasb_infer` resumable cache (serialize,
contiguity heal, manifest invalidation, atomic writes, `default_cache_dir`) |
303     | `test_eval_metrics.py` | `eval.metrics` IoU/matching/score/pr_curve |
304     | `test_eval_annotations.py` | `eval.annotations` GT CSV parse/validate,
`write_scaffold` |
305     | `test_eval_harness.py` | `eval.harness` DB-pred fetch + evaluate + report_to_dict;
also `@run_eval.py::main`/`get_file_md5` (exit codes, `--scaffold`/`--json`) |
306     | `test_eval_batch.py` | `eval.batch` path resolve / stage select / aggregate
(micro/macro, zero-div) |
307     | `test_calibration.py` | `eval.calibration`
```

```
pct_improvement/kfold/select_best/cross_validate/LOVO/improvement_report |
308     | `test_calibrate_wasb.py` | `eval.calibrate_wasb`
load_golden_gts/make_predict_fn/grid/run, BASELINE |
309     | `test_calibrate_local.py` | `eval.calibrate_local` local grid + crossing gate, leave-
one-video-out |
310     | `test_export_wasb_segments.py` | `eval.export_wasb_segments` segment/comparison CSV
schemas, mmss; export->slicer reuse |
311     | `test_eval_training.py` | `eval.training` label_candidates / feature extraction /
build_training_set |
312     | `test_eval_classifier.py` | `eval.classifier.LogisticRegression`
fit/predict_proba/balanced/JSON round-trip |
313     | `test_eval_regression.py` | `eval.regression` baseline store/compare/gate; also
`@run_regression.py::main` (exit-code CI gate) |
314     | `test_distill_local.py` | `eval.distill_local` build_candidates / predict_rallies /
learned-vs-baseline |
315     | `test_gemini_refine.py` | `eval.gemini_refine` merge_overlaps / _offset_segments (pure
helpers, no API) |
316     | `test_hit_detector.py` | `pipeline/audio/hit_detector` (detect_hits, onset_envelope,
_moving_stats) + `eval/audio/e1_probe.cluster_hits` |
317     | `test_rally_slicer.py` | `tools.rally_slicer.compute_clip_windows` (pure) + label
parse |
318     | `test_video.py` | `utils.video` duration/chunk (subprocess mocked) |
319     | `test_geometry.py` | `utils.geometry` center/distance/velocity/normalised-velocity/IoU
|
320     | `test_database.py` | `infrastructure.database.Database`
schema/CRUD/migrations/`user_version`/`EXPECTED_CANDIDATE_COLUMNS` guard |
321     | `test_config.py` | `config` (load_config defaults, save_default_config, AppConfig
defaults pinned, `extra` tolerance, `.get()` dict-compat) |
322     | `test_reporting_harvest.py` | `reporting` (`emit_indexer_report`,
`harvest.record_run`); ? `importorskip("obsreport")`; all 7 footgun rules + plain-dict-OR-
AppConfig + `report.enabled=False` |
323     | `test_reporting_spec_faq.py` | `reporting/spec.yaml` <-> `README.md` cross-ref
integrity (every `faq_ref` resolves; actionable rules cite a faq_ref); ?
`importorskip("obsreport")` |
324     | `test_validators_base.py` | `pipeline/validators/base.ValidationRegistry`
(register/run_all/`run_all_with_context` -> `(results,context)`; exception captured not raised)
|
325     | `test_sport_profiles.py` | `sports` registry + BadmintonSportProfile (prompt contains
"decimal seconds"/chunk value; relevance config keys) |
326     | `test_providers.py` / `test_provider_gemini.py` | `providers` registry +
GeminiProvider lifecycle (`MAX_UPLOAD_MB` guard, non-list JSON -> [], cleanup delete) |
327     | `test_annotation_providers.py` / `test_automated_provider.py` | `annotations`
registry, FileProvider, AutomatedProvider (oracle/lineage); keystone one-scorer contract ->
`eval.metrics.score` |
328     | (no dedicated test) | individual `pipeline/validators/*` impls,
`pipeline/stitcher.compiler`, `main.py`, `run_process_and_stitch.py`,
`pipeline/detectors/wasb_runner`, `pipeline/segmenters/{wasb_hybrid,motion}`,
`eval/audio/e1_probe` (diagnostic) ? ? exercised only indirectly |
329     | ? stale `.pyc` only | `tests/__pycache__/test_gemini_api.*` ? no live source exists |
330
```

7. user (2026-06-10T06:07:22.784Z)

```
No matches found
```

8. user (2026-06-10T06:08:01.906Z)

```
Structured output provided successfully
```

9. assistant (2026-06-10T06:08:13.303Z)

```
Verification complete.
```